

# Active Directory LDAPS Certificate Selection – Michael Waterman

---



michaelwaterman.nl/2026/02/03/deep-dive-active-directory-ldaps-certificate-selection

Michael Waterman

February 3, 2026

In my previous blog about [LDAPS certificates](#), I briefly touched on a topic that often leads to confusion, how a Domain Controller actually decides *which* certificate to use for LDAPS. At the time, I promised to dive deeper into that specific mechanism, because understanding it is critical when troubleshooting seemingly “mysterious” LDAPS issues. This post is that promised deep dive.

Many administrators assume that as long as a Domain Controller has a valid certificate with Server Authentication, LDAPS will simply use it. In reality, the selection process is far more complicated. A Domain Controller can easily have multiple certificates that technically meet all requirements, yet Windows will still pick only one, and not always the one you expect. The result can be intermittent connection errors, name mismatches, or sudden failures after certificate renewals.

To truly control LDAPS behavior, it's essential to understand the priority order Windows follows when selecting a certificate. In this article, I'll explain that process in detail, let's dive in.

## How Windows selects an LDAPS certificate

---

When a client connects to a Domain Controller over LDAPS, Windows must choose a single certificate to present during the TLS handshake. This choice is automatic and based on two phases: *validation* and *selection*.

In the validation phase, only certificates that meet the [LDAPS requirements](#) are considered. At a minimum, the certificate must have an associated private key, include the *Server Authentication* Enhanced Key Usage (*OID 1.3.6.1.5.5.7.3.1*), and match the Domain Controller's identity (typically through the *Subject Name* or *Subject Alternative Name*). These baseline requirements are widely referenced and form the “gate” that determines whether a certificate is eligible at all.

Once Windows has a set of eligible certificates, the selection phase starts. The most misunderstood detail is that Windows does not pick a certificate randomly. Instead, after validating candidates, it selects *the certificate with the most distant expiration date in the future* (the furthest *NotAfter* value). In other words: if you have multiple valid Server Authentication certificates that all meet the matching rules:

┆ *The one that stays valid the longest, wins.*

Where this becomes operationally relevant is the priority order of certificate stores. If a suitable certificate is present in the Active Directory Service Certificate Store (*NTDS\Personal* store), it takes precedence over certificates in the regular *Local Computer\Personal (MY)* store. If there is no usable certificate in NTDS, Windows falls back to searching the computer store. This matters because it's common for Domain Controllers to accumulate multiple Server Authentication certificates over time (autoenrollment, service certificates, renewals, migrations), and that can lead to unexpected LDAPS behavior unless you take control.

The practical takeaway is simple: if you want controlled LDAPS behavior in environments where multiple eligible certificates exist, placing the intended LDAPS certificate in the NTDS store ensures it wins the selection process every time. I'll show you how this can be accomplished, both manually and in an automated fashion.

## NTDS Store vs Computer Store

---

Windows systems normally store certificates in the "*Local Computer\Personal*" (MY) store, and most people are familiar with this concept. Domain Controllers, however, have an additional and lesser-known location: the "*NTDS\Personal*" store. This store exists specifically for Active Directory Domain Services and plays a key role in LDAPS certificate selection.

The practical rule is simple:

- If a valid certificate is present in "*NTDS\Personal*", it will always be used for LDAPS, if the requirements are met.
- Certificates in "*Local Computer\Personal*" are only considered when the NTDS store does not contain a suitable certificate.

This distinction becomes important when multiple certificates with Server Authentication exist on a Domain Controller. Over time, Domain Controllers often accumulate certificates through autoenrollment, Windows must choose between them based on general selection rules, which can lead to unexpected behavior.

To avoid production failures, Microsoft recommends placing the intended LDAPS certificate in the NTDS store. Doing so forces selection and ensures the correct certificate is always used. In short:

*the NTDS store is the administrator's way of taking explicit control over LDAPS certificate priority.*

## The downside of using the NTDS store

---

There is a significant downside to using the previously mentioned NTDS store. Put simply, auto-enrollment will stop managing the certificate once it is stored in the NTDS store. In other words, the certificate will not automatically renew through the standard auto-enrollment process.

This means you must have a process in place to manually renew or replace the certificate on each domain controller before it expires. I know, it's not ideal, but that's the trade-off that comes with explicitly controlling which certificate Active Directory uses.

If you are comfortable with that operational overhead, then using the NTDS store is still a perfectly valid approach. Just be aware of the lifecycle implications before continuing.

## Create the certificate template

---

When multiple eligible LDAPS certificates exist on a Domain Controller, the safest way to enforce the correct selection is to explicitly place the desired certificate in the “*NTDS\Persona*” store. Unlike the regular computer store, this location is not managed by initial autoenrollment and must be populated manually.

The process consists of three basic steps: exporting the intended certificate with its private key, opening the NTDS certificate store through the Certificates MMC snap-in, and importing the certificate into that store. Let's build this from scratch! I'll show you how to do it via the UI first, so you'll get a feeling on what it looks like and a PowerShell method next, great for automation.

**Note!** These first set of steps cannot be done on a Server Core, not even with the [Application Compatibility Features on Demand](#) pack. Use the PowerShell method described below for installation on a Windows Server Core.

- Open the Certificate Authority Management Console and open select “*Manage Templates*” on the “*Certificate Templates*” node. Duplicate the “*Kerberos Authentication*” template.
- On the “*General*” tab, provide a new name, e.g. “*NTDS Store Kerberos Authentication*”.
- On the “*Compatibility*” tab, select the highest available, server and client systems, with a minimum of server 2012 R2.
- On the “*Security*” tab, remove the “*Autoenroll*” permission for these types:
  - Enterprise Read-only Domain Controllers
  - Domain Controllers
  - Enterprise Domain Controllers
- On the “*Request Handling*” tab, select “*Allow private key to be exported*”

Properties of New Template

|                      |         |                  |                       |                 |
|----------------------|---------|------------------|-----------------------|-----------------|
| Subject Name         |         | Server           | Issuance Requirements |                 |
| Superseded Templates |         | Extensions       |                       | Security        |
| Compatibility        | General | Request Handling | Cryptography          | Key Attestation |

Purpose: Signature and encryption

☐ Delete revoked or expired certificates (do not archive)

☐ Include symmetric algorithms allowed by the subject

☐ Archive subject's encryption private key

☐ Authorize additional service accounts to access the private key

Key Permissions...

☒ Allow private key to be exported

☐ Renew with the same key

☐ For automatic renewal of smart card certificates, use the existing key if a new key cannot be created

Do the following when the subject is enrolled and when the private key associated with this certificate is used:

☒ Enroll subject without requiring any user input

☐ Prompt the user during enrollment

☐ Prompt the user during enrollment and require user input when the private key is used

OK Cancel Apply Help

On the “Subject Name” tab, select “Supply in the request” leave “Use subject information from existing certificates for auto-enrollement renewal requests” unchecked. Auto-enrollment will not work anyway.

NTDS Store Kerberos Authentication Properties

Superseded Templates Extensions Security Server

General Compatibility Request Handling Cryptography Key Attestation

Subject Name Issuance Requirements

☒ Supply in the request

☐ Use subject information from existing certificates for autoenrollment renewal requests

☐ Build from this Active Directory information

Select this option to enforce consistency among subject names and to simplify certificate administration.

Subject name format:

None

☐ Include e-mail name in subject name

Include this information in alternate subject name:

☐ E-mail name

☐ DNS name

☐ User principal name (UPN)

☐ Service principal name (SPN)

OK Cancel Apply Help

- On the “*Issuance Requirements*” tab, select “*CA Certificate manager approval*”
- Click “OK” and issue the certificate on your CA Server

Enable Certificate Templates

Select one Certificate Template to enable on this Certification Authority.

Note: If a certificate template that was recently created does not appear on this list, you may need to wait until information about this template has been replicated to all domain controllers.

All of the certificate templates in the organization may not be available to your CA.

For more information, see [Certificate Template Concepts](#)

| Name                               | Intended Purpose                                               |
|------------------------------------|----------------------------------------------------------------|
| Kerberos Authentication            | Client Authentication, Server Authentication, Smart Card Logon |
| Key Recovery Agent                 | Key Recovery Agent                                             |
| LDAPS                              | Server Authentication                                          |
| NTDS Store Kerberos Authentication | KDC Authentication, Smart Card Logon, Server Authentication    |
| OCSP Response Signing              | OCSP Signing                                                   |
| RAS and IAS Server                 | Client Authentication, Server Authentication                   |
| Router (Offline request)           | Client Authentication                                          |
| Smartcard Logon                    | Client Authentication, Smart Card Logon                        |
| Smartcard User                     | Secure Email, Client Authentication, Smart Card Logon          |

OK Cancel

## Option 1 – Certificate placement via the GUI

---

Now that the certificate template is configured, following a process very similar to the one described in my previous blog, it's time to request the certificate and install it in the *"Local Computer\My store"* on the Domain Controller. You might be wondering: *"Aren't we supposed to place the certificate in the NTDS store instead?"* Don't worry, we'll get there. For now, just follow along with the initial steps.

It's important to note that the GUI-based approach described here will not work on Windows Server Core, not even with the Application Compatibility Feature on Demand package installed, as it does not include certificate management tools. All steps in Option 1 require a Domain Controller running the full Desktop experience.

### Request the certificate

---

- Open *"certlm.msc"*
- Right click on *"Personal"*, click *"All Tasks"*, *"Request New Certificate"*
- Click *"Next"* on the *"Before You Begin"* page
- Click *"Next"* on the *"Select Certificate Enrollment Policy"* page, make sure the default *"Active Directory Enrollment Policy"* is selected
- Select the *"NTDS Store Kerberos Authentication"* and click on *"More information is required to enroll for this certificate"*
- Add these settings on the subject tab:
  - Subject name:  
*Common name*: FQDN of your Domain Controller, e.g: lab-dc-03.corp.michaelwaterman.nl
  - Alternative name:
    - *DNS*: FQDN of your Domain Controller, e.g: lab-dc-03.corp.michaelwaterman.nl
    - *DNS*: The full domain name, e.g: FQDN of your Domain Controller, e.g: corp.michaelwaterman.nl
    - *DNS*: Load balancer FQDN, e.g: ldaps.corp.michaelwaterman.nl
- Click *"OK"*, Click *"Enroll"*, Click *"Finish"*

### Approving the request

---

- On your CA server, open *"Certification Authority"*
- Open the *"Pending Requests"*
- Right click on the *"Request ID"* itself and click *"All Tasks"*, *"Issue"*

### Retrieving and installing the certificate

---

Unfortunately there is no way in the UI to retrieve an issued certificate, well at least not that I'm aware of. Let's use the *"certutil"* command-line tool to retrieve and install the certificate.

On your Domain Controller, open an elevated command-prompt. Make sure you don't have PowerShell selected. Use the following command:

This will retrieve the certificate and place it into the local computers "MY" store.

## Exporting the certificate

---

The next step is to export the certificate to a portal format (PFX). Just follow along.

- Open "certlm.msc"
- Browse to "Personal", "Certificates", right click on the installed certificate, select "All Tasks", "Export".
- Click "Next" on the "Welcome to the Certificate Export Wizard"
- Select "Yes, export the private key", click "Next". Please note that we explicitly enabled this option in the template.
- On the "Export file format" page, select only these options:
  - Delete the private key if the export is successful
  - Enable certificate privacy

### Export File Format

Certificates can be exported in a variety of file formats.

Select the format you want to use:

- ☐ DER encoded binary X.509 (.CER)
- ☐ Base-64 encoded X.509 (.CER)
- ☐ Cryptographic Message Syntax Standard - PKCS #7 Certificates (.P7B)
  - ☐ Include all certificates in the certification path if possible
- ☒ Personal Information Exchange - PKCS #12 (.PFX)
  - ☐ Include all certificates in the certification path if possible
  - ☒ Delete the private key if the export is successful
  - ☐ Export all extended properties
  - ☒ Enable certificate privacy
- ☐ Microsoft Serialized Certificate Store (.SST)

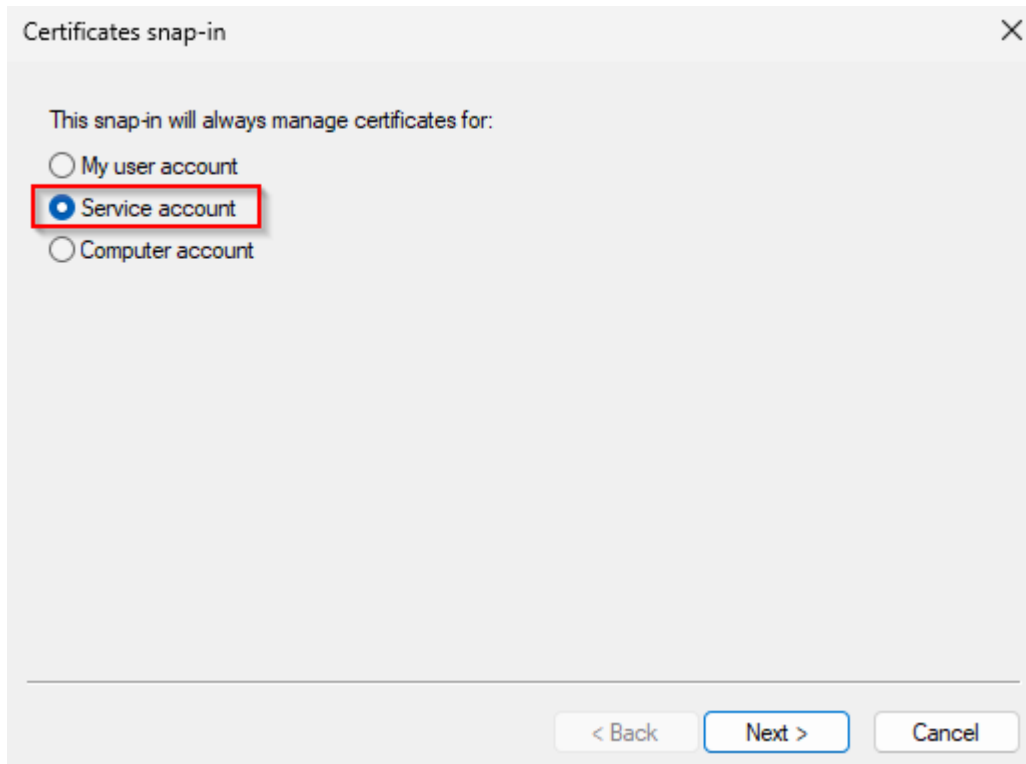
- Click "Next" after the selection.
- On the "Security" page, select "Password:", enter a password of your choosing. Select "AES256-SHA256" for the encryption type, click "Next"
- On the "File to export" page, browse to a location where you want to store the PFX file, click "Next", Click "Finish", click "OK"
- After a successful export, delete the certificate from the "Personal" store as it is no longer needed.

## Placing the certificate in the NTDS Store

---

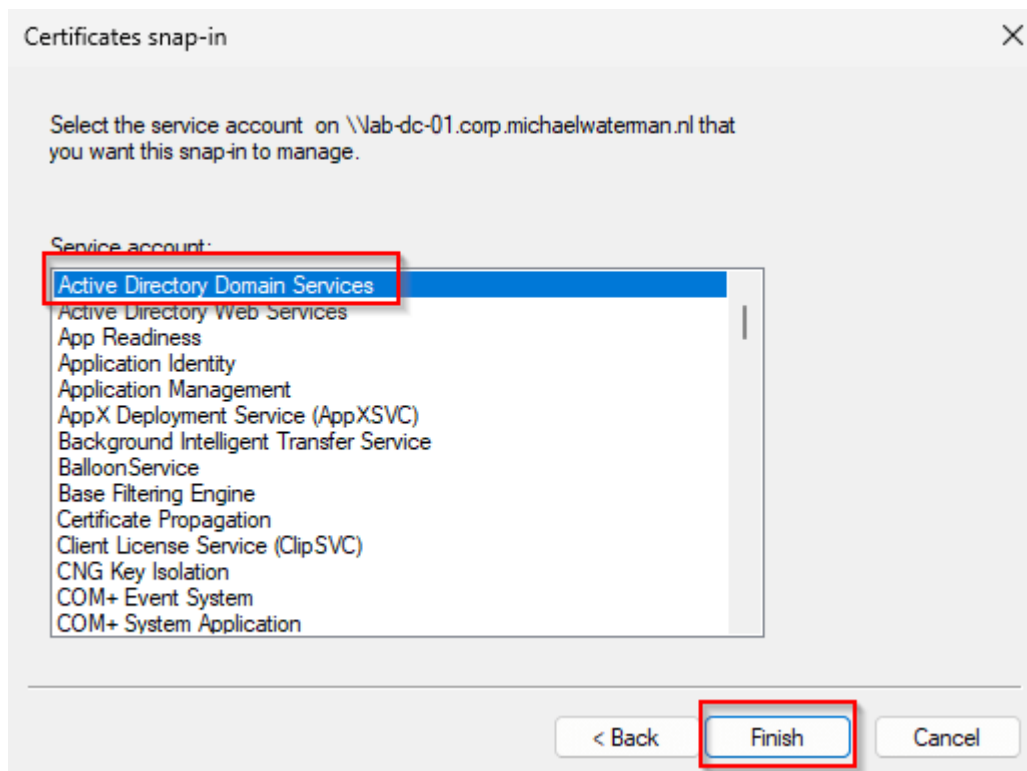
Now it's time to import the certificate into the Active Directory service. On your Domain Controller, open up your Management Console (MMC) and follow these steps.

- Click “File”, “Add/ Remove Snap-in”, select “Certificates”, click “Add”
- Select “Service account”, click “Next”



- Select “*Local computer*”, and click “Next”
- Select the “*Active Directory Domain Services*” service and click “*Finish*” and finally “OK”





- Browse to “*NTDS\Personal*”, right click and choose, “*All Tasks*”, “*Import*”
- On the “*Welcome to the Certificate Import Wizard*” page, click “*Next*”
- On the “*File to import*” page, browse to the location where you stored the exported PFX file. If you don’t see the file, select “*Personal information exchange*” in the drop-down box in the right bottom corner. Click “*Open*”, click “*Next*”
- On the “*Private key protection*” page, enter the password assigned to the PFX file, click “*Next*”
- On the “*Certificate store*” page, make sure that “*NTDS\Personal*” is listed, click “*Next*”, click “*Finish*”, click “*OK*”

And that’s it! Now let’s see if what we did actually works.

## Testing the certificate

Now that we have successfully imported the certificate into the “*NTDS\Personal*” store it’s time to give it a test run, we don’t just believe the theory, we need to validate it as well.

First we need to get the thumbprint of the certificate in the “*NTDS\Personal*” store, obviously we can do that with PowerShell, but we’re still in the GUI section, so I’m going to click. Just follow these few steps.

- Open the MMC console we created earlier for accessing the “*NTDS\Personal*” store.
- Browse to the personal store and right click on the imported certificate, select “*Open*”
- On the “*Details*” tab and browse to the last entry, it should say “*Thumbprint*”. Note the thumbprint listed. In my case it’s “*2e76235f3419a29b3becd388ae27f81320dd4e14*”

As for the validation part, I was about to make some PowerShell code but someone beat me to it, by a couple of years even. I'm reusing the [code found on this page](#). You really should read that page as well, great info! For the sake of completeness I'm listing the code I'm about to use here as well.

```
function Get-CertificateFromTlsHandshake {
    param(
        [Parameter(Mandatory=$true, Position=0)]
        [string] $HostName,
        [int] $Port = 443,
        [string] $Destination
    )

    $tcp = New-Object System.Net.Sockets.TcpClient($HostName, $Port)
    $state = @{}
    $ssl = New-Object System.Net.Security.SslStream($tcp.GetStream(), $false, {
        param($Sender, $Certificate, $Chain, $SslPolicyErrors)
        $state.cert = New-Object
System.Security.Cryptography.X509Certificates.X509Certificate2($Certificate)
        return $true
    })

    $ssl.AuthenticateAsClient($HostName)
    $ssl.Dispose()
    $tcp.Dispose()

    if (-Not [string]::IsNullOrEmpty($Destination)) {
        $AsByteStream = if ($PSEdition -eq 'Core') { @{AsByteStream = $true} } else
{ @{'Encoding' = 'Byte'} }
        Set-Content -Path $Destination -Value $cert.GetRawCertData() @AsByteStream
    }

    return $state.cert
}
```

Inject this code, or function I should say on your test machine. In my case I'm using a Windows 11 machine. Next I'm running this PowerShell command:

```
Get-CertificateFromTlsHandshake -HostName lab-dc-03.corp.michaelwaterman.nl -Port 636
```

The output will list the thumbprint of the certificate that will now be used for LDAPS! If all went well, it will match the thumbprint I listed above. Cool stuff right!

```
PS C:\Users\T0-ADM-01> Get-CertificateFromTlsHandshake -HostName lab-dc-03.corp.michaelwaterman.nl -Port 636
```

| Thumbprint                               | Subject                              |
|------------------------------------------|--------------------------------------|
| 2E76235F3419A29B3BECD388AE27F81320DD4E14 | CN=lab-dc-03.corp.michaelwaterman.nl |

## Option 2 – Using PowerShell, this is the way...

---

Option 1 for creating, exporting, and importing a certificate involved quite a lot of manual clicking. While it's great for understanding what actually happens behind the scenes, repeating those steps multiple times quickly becomes annoying. More importantly, as mentioned earlier, this GUI-based procedure simply isn't possible on Windows Server Core due to the lack of local certificate management tools.

Even if a graphical interface were available, this approach wouldn't scale well in production environments. For these systems, you want a method that is predictable, repeatable, and easy to automate. Fortunately, we have PowerShell remote management to solve exactly that problem.

In this section, I'll walk through how to perform the exact same steps as in Option 1, requesting, exporting, and importing the certificate, but in a way that can be fully executed remotely and managed efficiently at scale.

### Configuring the certificate

---

On your management workstation (P.A.W for example) open up a PowerShell command shell. Use the following steps:

Connect to the Domain Controller over PowerShell remote management

```
Enter-PSSession -ComputerName "lab-dc-03.corp.michaelwaterman.nl"
```

Replace the hostname with your own server name.

Request the certificate

```
$enroll = Get-Certificate `
    -Template "NTDSStoreKerberosAuthentication" `
    -SubjectName "cn=lab-dc-03.corp.michaelwaterman.nl" `
    -DnsName lab-dc-03.corp.michaelwaterman.nl, ldaps.corp.michaelwaterman.nl,
corp.michaelwaterman.nl `
    -CertStoreLocation "cert:\LocalMachine\My"
```

- As with the previous option, issue the certificate on your CA server. See option one for the exact steps.
- Retrieve the request and install it, please not that you can also use "*certutil -pulse*" at this stage, but we're sticking to PowerShell.

```
$request = Get-ChildItem Cert:\LocalMachine\Request\$( $enroll.Request.Thumbprint)
Get-Certificate -Request $request
```

**Tip!** If you would like to know the Thumbprint of the certificate we just installed, use this PowerShell function I've cooked up to see all the certificates created with the specific template I created:

```
function Get-CertThumbprintByTemplateName {
    [CmdletBinding()]
    param(
        [Parameter(Mandatory, Position = 0)]
        [string] $TemplateNameContains
    )

    $templateInfoOid = "1.3.6.1.4.1.311.21.7" # Certificate Template Information

    Get-ChildItem -Path Cert:\LocalMachine\My | ForEach-Object {
        $cert = $_

        $ext = $cert.Extensions | Where-Object { $_.Oid.Value -eq $templateInfoOid }

        if (-not $ext) { return }

        $raw = $ext.Format(0)

        $m = [regex]::Match($raw, '^Template=(.+?)\(')
        if (-not $m.Success) { return }

        $templateName = $m.Groups[1].Value.Trim()

        if ($templateName -like "$TemplateNameContains") {

            # Clean up the PSPATH to a more readable format
            $cleanPath = $cert.PSPath -replace
"Microsoft.PowerShell.Security\Certificate:", ""

            [PSCustomObject]@{
                Subject      = $cert.Subject
                Template     = $templateName
                Thumbprint    = $cert.Thumbprint
                NotAfter     = $cert.NotAfter
                StorePath     = $cleanPath
            }
        }
    }
}
```

To use it, simply execute the following command:

```
Get-CertThumbprintByTemplateName "NTDS Store Kerberos Authentication"
```

```
[lab-dc-03.corp.michaelwaterman.nl]: PS C:\> Get-CertThumbprintByTemplateName "NTDS Store Kerberos Authentication"

Subject       : CN=lab-dc-03.corp.michaelwaterman.nl
Template      : NTDS Store Kerberos Authentication
Thumbprint    : 530DA33E581F7C461F8ABFEDDF042375DF12E315
NotAfter      : 3-3-2026 13:30:27
StorePath     : LocalMachine\My\530DA33E581F7C461F8ABFEDDF042375DF12E315
```

**Note!** Look at the Thumbprint of the certificate, I'll use it later to verify.

Just as with option one we will now need to export the certificate to a PFXfile format, so we can import it again into the NTDS store. Just use the following command:

```
$password = ConvertTo-SecureString -String "StrongPassword!" -Force -AsPlainText
Export-PfxCertificate `
  -Cert "cert:\LocalMachine\My\530DA33E581F7C461F8ABFEDDF042375DF12E315" `
  -FilePath "C:\Temp\ldaps.pfx" `
  -Password $password `
  -Force
```

**Note!** Replace the password with your own strong password.

**Note!** Make sure that the directory where the PFX is stored already exists

As we no longer need the certificate in the servers personal certificate store we can delete it. Use the following command:

```
Remove-Item -Path Cert:\LocalMachine\My\128A24F0495198820209B6B2CC413DE6F69B68A1
```

Now comes the more challenging part, importing the certificate into the “*NTDS\Personal store*”. Unfortunately, PowerShell does not provide a direct certificate provider path that exposes the certificate stores of specific services. This means we can’t simply use a standard Cert:\ path like we can for the Local Machine store.

Fortunately, the excellent [deep-dive article](#) I referenced earlier provides the exact code required to perform this task programmatically. Rather than reinventing the wheel, I can use that approach to import the certificate into the NTDS store in a fully automated way. For convenience, I’ve included the necessary script below as well. Copy and paste it into the PowerShell command window.

```
Import-Module -Name Microsoft.PowerShell.Security

function Get-ServiceCertStore {
    [CmdletBinding()]
    param (
        [Parameter(Mandatory=$true, ValueFromPipeline=$true,
ValueFromPipelineByPropertyName=$true)]
        [string] $ServiceName,

        [Parameter()]
        [string] $Name = 'My',
```

```

        [Security.Cryptography.X509Certificates.OpenFlags]
        $openFlags = [Security.Cryptography.X509Certificates.OpenFlags]::MaxAllowed
    )

    begin {
        $typeParams = @{
            TypeDefinition = @'
using Microsoft.Win32.SafeHandles;
using System;
using System.Runtime.InteropServices;

namespace X509
{
    public class NativeMethods
    {
        [DllImport("Crypt32.dll")]
        public static extern bool CertCloseStore(
            IntPtr hCertStore,
            uint dwFlags);

        [DllImport("Crypt32.dll", CharSet=CharSet.Unicode, SetLastError=true)]
        public static extern SafeX509Store CertOpenStore(
            IntPtr lpszStoreProvider,
            uint dwEncodingType,
            IntPtr hCryptProv,
            uint dwFlags,
            string pvPara);
    }

    public class SafeX509Store : SafeHandleZeroOrMinusOneIsInvalid
    {
        public SafeX509Store() : base(true) { }

        protected override bool ReleaseHandle()
        {
            return NativeMethods.CertCloseStore(handle, 0);
        }
    }
}
'@
    }
    Add-Type @typeParams

    $provider = [IntPtr]::new(10)
    $flags = (0x00050000 -bor 0x00000004)
    $flagType = [Security.Cryptography.X509Certificates.OpenFlags]

    $openMode = [int]$openFlags -band 3
    $accessFlags = switch ($openMode) {
        0 { 0x00008000 }
        2 { 0x00001000 }
    }

```

```

        default { 0 }
    }
    $flags = $flags -bor $accessFlags

    if ($OpenFlags.HasFlag($flagType::OpenExistingOnly)) {
        $flags = $flags -bor 0x00004000
    }
    if ($OpenFlags.HasFlag($flagType::IncludeArchived)) {
        $flags = $flags -bor 0x00000200
    }
}

process {
    $handle = [X509.NativeMethods]::CertOpenStore(
        $provider, 0, [IntPtr]::Zero, $flags, "$ServiceName\$Name"
    ); $err = [Runtime.InteropServices.Marshal]::GetLastWin32Error()

    if ($handle.IsInvalid) {
        $exp = [ComponentModel.Win32Exception]$err
        Write-Error -Message "Failed to open '$ServiceName\$Name':
        $($exp.Message)" -Exception $exp
        return
    }

    try {

[Security.Cryptography.X509Certificates.X509Store]::new($handle.DangerousGetHandle(
))
    }
    finally {
        $handle.Dispose()
    }
}
}

```

Import the PFX file into the “*NTDS\Personal store*” using these commands, adjust them to reflect your situation.

```

$CertificateFile = Resolve-Path "C:\Temp\ldaps.pfx"
$CertificatePassword = ConvertTo-SecureString -String "StrongPassword!" -Force -
AsPlainText
$KeyStorageFlags =
[System.Security.Cryptography.X509Certificates.X509KeyStorageFlags]'MachineKeySet,
PersistKeySet'
$Certificate =
[System.Security.Cryptography.X509Certificates.X509Certificate2]::new($CertificateF
ile, $CertificatePassword, $KeyStorageFlags)
$CertificateThumbprint = $Certificate.Thumbprint

$CertificateStore = Get-ServiceCertStore -ServiceName "NTDS" -Name "My"
$CertificateStore.Add($Certificate)

```

To quickly see if the certificate is indeed successfully imported and contains the private key, use this code:

```
$CertificateStore.Certificates | Where {$_.Thumbprint -eq  
"530DA33E581F7C461F8ABFEDDF042375DF12E315"} | select -ExpandProperty PrivateKey
```

If you see something like the data below, the import is successful!

```
PublicOnly           : False  
CspKeyContainerInfo  : System.Security.Cryptography.CspKeyContainerInfo  
KeySize              : 2048  
KeyExchangeAlgorithm : RSA-PKCS1-KeyEx  
SignatureAlgorithm   : http://www.w3.org/2000/09/xmldsig#rsa-sha1  
PersistKeyInCsp      : True  
LegalKeySizes        : {System.Security.Cryptography.KeySizes}
```

And that's it! I've also created, imported, exported and imported the key into the "NTDS\Personal" store using PowerShell!

Last step is to validate that the key is actually used. We already know the thumbprint (530DA33E581F7C461F8ABFEDDF042375DF12E315) this time. Let's use the same function as before. Logon to a workstation and run the following command:

```
Get-CertificateFromTlsHandshake -HostName lab-dc-03.corp.michaelwaterman.nl -Port  
636
```

This will result in the correct thumbprint being shown, proving that the PowerShell commands I showed you have the exact same effect as the GUI walk-through. You're now able to automate the majority of these steps or at the same time, if you have just a few Domain Controllers, run it manually, it's up to you.

## What I've learned along the way

---

In my [previous blog](#), I walked through the basics of configuring certificates for LDAPS and explained the technical requirements needed to make it work. At the time, I promised to dive deeper into how Windows actually selects an LDAPS certificate, and with this article, I feel I've finally completed that story.

What I've learned is that getting LDAPS right isn't just about issuing a valid certificate. It's about understanding how certificate selection really works, how store priority affects that process, and how easily things can go wrong when multiple Server Authentication certificates are present. Placing the intended certificate in the *NTDS store* turned out to be the key to making LDAPS behavior predictable.



I've also learned that real-world management matters just as much as theory. GUI steps are great for learning and troubleshooting, but in environments running Server Core or managed remotely, PowerShell remote management becomes essential. Being able to automate the entire process, from requesting the certificate to importing it into the correct store, makes the solution scalable and reliable.

Looking back at both blogs together, I've now shown the full picture, how to configure LDAPS correctly, how to make it highly available, and how to ensure the right certificate is always used in the right store. With these pieces in place, LDAPS is no longer something mysterious or fragile, it becomes just another well-understood and manageable part of Active Directory.

And knowing is half the battle...

Until next time, as always feedback is very welcome!